CS61C Spring 2025 — Yan Midterm

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT the Name and Student ID of the person to your left: _____

PRINT the Name and Student ID of the person to your right: _____

PRINT the Name and Student ID of the person in front of you: _____

PRINT the Name and Student ID of the person behind you: _____

You have 110 minutes. There are 8 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	8	Total
Points:	14	11	21	24	13	8	9	0	100

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◌✓ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

If an answer requires hex input, you must only use capitalized letters (`0xB0BACAFE` instead of `0xb0bacafe`). For hex and binary, please include prefixes in your answers unless otherwise specified, and do not truncate any leading 0's. For all other bases, do not add any prefixes or suffixes.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam.

Acknowledge that you have read and agree to the honor code above and sign your name below:

_____

Clarifications made during the exam:

Q1.5: On line 6, jal ra should be jr ra [fixed]

Q5.4: Main memory access time refers to the L1 cache miss penalty.

Q4.13: On page 9, the example under the table should read "For example, if a0 points to … then `store_str_as_int` would store the word `0x12345678` at `0x10000000`" [fixed]

Q1 *Potpourri*  **(14 points)**

Q1.1 (1.5 points) Convert −49 from decimal to 8-bit binary two's complement representation.

`0b`

Q1.2 (1.5 points) Convert the biased hexadecimal number `0x2A` (with a bias of −31) to decimal.

Q1.3 (2 points) Convert the RISC-V instruction `beq a3 s1 12` into 32-bit hexadecimal machine code.

`0x`

Q1.4 (2 points) Convert the following RISC-V machine code into its corresponding instruction. If there is an immediate value, express it in decimal form. Provide the appropriate register names, not numbers, where necessary (e.g., `s5` instead of `x21`).

`0x01685A93`

Q1.5 (2 points) In the code below, which registers are used in a way that **violates** calling convention?

```
1 foo:
2   lui a0, 0xFFEE0
3   addi t0, a0, 4
4   jal ra bar
5   lw s0, 0(t0)
6   jr ra
```

- ☐ `a0`
- ☐ `t0`
- ☐ `sp`
- ☐ `a1`
- ☐ `s0`
- ☐ `ra`

Q1.6 (1 point) Which of these statements about the Assembly stage of CALL are true? Select all that apply.

- ☐ Outputs a file that contains text and data segments
- ☐ Converts high-level language code to assembly language code
- ☐ Generates a relocation table
- ☐ Resolves all label and data references

(4 points) You have started a GDB session with the following code segment:

```
1    int main(int argc, char *argv[]) {
2      char *crops[4] = {"parsnip", "melon", "pumpkin", "corn"};
3      int count = // code omitted;
4  --> quality(crops, count);
5    }
6
7    void quality(char** crops, int count){
8      while (count >= 0) {
9        // code omitted
10       quality_crops = // code omitted
11       count -= 1;
12     }
13     return quality_crops
14   }
```

Your GDB session is about to execute line 4, but has not executed it yet. Write a sequence of GDB commands that would perform the following actions. Assume each command is executed before the next:

```
# 1. print the address of count:

(gdb) _____
                                        Q1.7
# 2. set a breakpoint on line 10 if count is even:

(gdb) _____
                                        Q1.8
# 3. go to your breakpoint without restarting your session:

(gdb) _____
                                        Q1.9
# 4. print 1 if quality_crops is 3, otherwise print 0:

(gdb) _____
                                        Q1.10
```

Q2 *Conditional Eggsecution* 🥚 **(11 points)**

For the entirety of this question, consider the following C code:

`conditional_sum`: Takes the values greater than or equal to `50` in `arr`, and adds their sum to `*dest`.		
Arguments	`int32_t *dest`	Pointer to allocated space for the sum.
	`int32_t *arr`	Array of integers to conditionally sum.
	`size_t length`	The number of elements in `arr`.
Return value	`void`	

```
1 void conditional_sum(int32_t *dest, int32_t *arr, size_t length) {
2   for (int i = 0; i < length; i++) {
3     if (arr[i] >= 50) {
4       *dest += arr[i];
5     }
6   }
7 }
```

Q2.1 (4 points) We decide to call this function from `main` on a 32-bit **little-endian** system:

```
1 int main() {
2   int32_t dest1 = 0;
3   int32_t dest2 = 0;
4   int16_t arr1[4] = {0x0080, 0xA000, 0x0036, 0x0000};
5   uint8_t arr2[4] = {0x6C, 0xFF, 0xFF, 0xFF};
6   conditional_sum(&dest1, (int32_t *) arr1, 2);
7   conditional_sum(&dest2, (int32_t *) arr2, 1);
8 }
```

What are the values in `dest1` and `dest2` in hexadecimal after `main` is run?

`dest1: 0x`

`dest2: 0x`

Q2.2 (1 point) Which section of memory would the variable `dest1` live in?

◯ Stack ◯ Heap ◯ Code ◯ Static

Q2.3 (6 points) Write the loop body of `conditional_sum` to use **bit masking** instead of branch statements (`if-else`) or comparisons.

Hint: If `arr[i] < 50`, what is the sign of `arr[i] - 50`?

You may assume that:

- Signed integers are stored in two's complement.
- No overflows occur during addition/subtraction. This means that `arr[i] - 50` does not overflow.
- `dest` is large enough to hold the result.

Your answer may consist of only the following operations:

Allowed	
Addition / Subtraction	`+ -`
Shifts	`<<` and `>>` (sign-extended)
Bitwise Operations	`~ & | ^`
Boolean Operations	`! && ||`
Parentheses	`()`
Array Subscripting	`arr[x]`

```
1 void conditional_sum(int32_t *dest, int32_t *arr, size_t length) {
2   uint32_t mask;
3   for (int i = 0; i < length; i++) {

4     mask = ____________________Q2.3____________________;
5     *dest += mask & arr[i];
6   }
7 }
```

Q3 *Generic Linked Lists* (21 points)

This question defines a linked list with a twist: The linked list uses generics to store data without a data type! Consider the following definition of a linked list node, `node_t`:

```
1 typedef struct node {
2   struct node *next_node; // next_node is a node_t*. If this node is the end
3                           // of the list, next_node is NULL
4   size_t width;           // size of data, in bytes
5   void *data;
6 } node_t;
```

(9 points) Implement the function `add_head_node`. The `memcpy` function prototype is provided below for your reference:

```
void *memcpy(void *dest, void *src, size_t n);
```

`add_head_node`: Add a new node to the head of `list` whose `data` member points to a heap-allocated copy of `temp`, which is `width` bytes long.		
Arguments	`node_t *list`	Pointer to linked list to add a new head node to
	`size_t width`	The size of `temp`, in bytes
	`void *temp`	A pointer to the data that should be copied into this node
Return value	`node_t *head`	The new head of the linked list

```
1 node_t* add_head_node(node_t *list, size_t width, void* temp){
2   node_t *head = ____Q3.1____;
3   head->next_node = ____Q3.2____;
4   head->width = ____Q3.3____;
5   ____Q3.4____;
6   memcpy(____Q3.5____, ____Q3.6____, ____Q3.7____);
7   return ____Q3.8____;
8 }
```

(Question 3 continued...)

(7 points) Implement the function **arr_to_ll** that uses **add_head_node** to convert an array of strings into a linked list, preserving the order of elements. In other words, the head of the linked list returned from **arr_to_ll** should contain a pointer to a copy of the data in **arr[0]** on the heap.

Regardless of what you wrote before, assume that **add_head_node** is correctly implemented.

arr_to_ll: Given a **char*** array, create an equivalent linked list of strings		
Arguments	**char** arr**	Array of strings
	size_t count	The number of strings in the array **arr**
Return value	**node_t *head**	The head of the linked list

```
1  node_t* arr_to_ll(char** arr, size_t count){
2    node_t *head = NULL;
3    for (size_t i = count; ____Q3.9____; ____Q3.10____) {
4      size_t len = ____Q3.11____;
5      head = add_head_node(____Q3.12____, ____Q3.13____, ____Q3.14____)
6    }
7    return ____Q3.15____;
8  }
```

(5 points) Finally, implement the function **free_ll** that frees a linked list returned from **add_head_node**. Assume that the **data** member of each node in the list points to heap-allocated memory.

free_ll: Given a linked list of nodes, free the entire linked list		
Arguments	**node_t *list**	Pointer to the linked list to free
Return value	**void**	

```
1  void free_ll(node_t* list){
2    node_t *curr = list;
3    while (____Q3.16____) {
4      node_t *temp = ____Q3.17____;
5      curr = ____Q3.18____;
6      free(____Q3.19____);
7      free(____Q3.20____);
8    }
9  }
```

Q4 *Not Like Us* 🗣️ **(24 points)**

Consider the function `store_first_byte` as described below on a 32-bit **little-endian** system:

`store_first_byte`: Reads the first two digits of a string as hexadecimal and stores it in memory.		
Arguments	`a0`	A `char*` consisting of `chars` from `'0'` – `'9'`. `strlen(a0)` is even and at least 2.
	`a1`	A **pointer** to allocated memory to store the first byte of the string pointed to by `a0` read as hexadecimal
Return value	`void`	

For example, if `a0` points to `"12345678"`, and `a1` contains `0x10000000`, then `store_first_byte` would store the byte `0x12` at `0x10000000`.

(10 points) Implement your own version of `store_first_byte` but the only load instruction you can use is `lbu`:

```
1  store_first_byte:
2      lbu t0 _____ (Q4.1)
3      _____ (Q4.2)
4      lbu t1 _____ (Q4.3)
5      andi t1 _____ (Q4.4)
6      _____ (Q4.5)
7      sb t0 _____ (Q4.6)
8      jr ra
```

(10 points) Now, re-implement `store_first_byte`, but the only load instruction you can use is `lhu`.

```
1  store_first_byte:
2      lhu t0 _____ (Q4.7)
3      srli t1 _____ (Q4.8)
4      andi t1 _____ (Q4.9)
5      _____ (Q4.10)
6      _____ (Q4.11)
7      sb t0 _____ (Q4.12)
8      jr ra
```

Now, consider the following function `store_str_as_int`:

`store_str_as_int`: Reads a string of digits as hexadecimal and stores it in memory.		
Arguments	`a0`	A `char*` consisting of `char`s from `'0'` – `'9'`. `strlen(a0)` is exactly 8.
	`a1`	A **pointer** to allocated memory to store the value of the string pointed to by `a0` read as hexadecimal
Return value	`void`	

For example, if `a0` points to `"12345678"`, and `a1` contains `0x10000000`, then `store_str_as_int` would store the word `0x12345678` at `0x10000000`.

Drake implements `store_str_as_int` in the following way:

```
1  store_str_as_int:
2      # prologue omitted
3  loop:
4      lb t0 0(a0)
5      beqz t0 end
6      # save a0 and a1 on the stack
7      jal ra store_first_byte
8      # restore a0 and a1 from the stack
9      addi a0 a0 2
10     addi a1 a1 1
11     j loop
12 end:
13     # epilogue omitted
14     ret
```

Regardless of what you wrote before, assume that `store_first_byte` works according to the specification.

Kendrick doubts that Drake's implementation of `store_str_as_int` works properly. To check Drake's implementation, Kendrick puts a pointer to `"12345678"` in `a0` and `0x10000000` in `a1`, then calls `store_str_as_int`. Assume Kendrick has already allocated sufficient memory at `0x10000000`.

Kendrick then executes `lw a0 0(t0)`, where `t0` holds the memory address `0x10000000`. After this instruction, Kendrick finds that `a0` holds the number `0x78563412`, not `0x12345678`.

Q4.13 (4 points) Why is Drake's implementation of `store_str_as_int` wrong? Give your answer in at most 20 words.

Q5 *Caches* 💸 **(13 points)**

Q5.1 (3 points) What is the tag-index-offset breakdown for a 32 KiB direct-mapped cache with 64B lines on a system with a 128KiB address space?

T: bit(s)	I: bit(s)	O: bit(s)

For Q5.2–Q5.3, the code below is executed on a 32-bit system with a 256B, 4-way set-associative cache with 8B cache lines and a first-in-first-out (FIFO) replacement policy. Assume the cache is cold prior to the start of loop 1.

```
1  #define STRIDE 2
2
3  // register int32_t <var> means that <var> is stored in a RISC-V register
4
5  void foo(int32_t *arr) {   // Assume arr = 0x1000 0020
6    register int32_t index;
7
8    /* LOOP 1 */
9    index = 0;
10   for (register int32_t j = 0; j < 32; j += 1) {
11     arr[index] = j;
12     index += STRIDE;
13   }
14
15   /* LOOP 2 */
16   index = 0;
17   for (register int32_t j = 0; j < 32; j += 1) {
18     arr[index] = j + 1;
19     index += STRIDE;
20   }
21 }
```

Q5.2 (4 points) How many cache hits and misses occur during `LOOP 2`?

Hits:	Misses:

Q5.3 (2 points) What is the smallest positive integer value for `STRIDE` such that every element accessed in `LOOP 1` and `LOOP 2` share the same index in the cache?

`STRIDE:`

Q5.4 (4 points) Our system uses a single L1 cache (L1$). Now, we run two separate programs to measure the performance of the cache and observe the following results:

Program A	
AMAT	200ns
L1$ Miss Rate	35%

Program B	
AMAT	100ns
L1$ Miss Rate	10%

What is our cache hit time and main memory access time? Assume these times remain consistent between Program A and Program B.

L1$ Hit Time: ns

Main Memory Access Time: ns

Q6 *Ba-lite-tro* ♥♣♦♠ **(8 points)**

Noah is playing the game Ba-lite-tro on his laptop. The game stores his score using the IEEE-754 single-precision floating-point format with 1 sign bit, 8 exponent bits (with a bias of −127), and 23 significand bits.

Q6.1 (2 points) Noah somehow scores −67.125 points?! Convert his score into hexadecimal in this format.

0x

Noah wants to switch to playing Ba-lite-tro on his phone, but it doesn't have enough storage! He decides to modify the game to use a **16-bit floating-point format** instead, following the IEEE-754 standard with 1 sign bit, 7 exponent bits (with a bias of −63), and 8 significand bits. This 16-bit format is used to store his game score.

Q6.2 (3 points) Given this new representation, what is the **maximum game score** (i.e. largest possible non-infinite positive number) in this format?

Express your answer first as 16-bit hexadecimal:

0x

Now express your answer in terms of powers of two and **simplify as much as possible.**

Noah decides to adjust the 16-bit game score format by changing how the 16 bits are allocated to the sign, exponent, and significand. He decides to reallocate bits to optimize for the largest maximum game score.

While adjusting, Noah still follows the IEEE-754 floating-point number format. This means that:

- There must be one sign bit
- The exponent bias is $-\left(2^{n-1}-1\right)$, where n is the number of exponent bits
- Infinities, NaN, and denormalized numbers are still encoded as expected.

Q6.3 (3 points) If Noah allocates his 16 bits optimally, what is the largest possible maximum game score?

Express your answer first as 16-bit hexadecimal:

0x

Now express your answer in terms of powers of two and **simplify as much as possible.**

Q7 *Boolean Algebra* 🦁🤼 **(9 points)**

NOT ($\overline{x}$), AND ($\cdot$), OR ($+$) each count as one operator. We will assume standard C operator precedence, so use parentheses when uncertain.

Your answers may consist of the following operators and symbols:

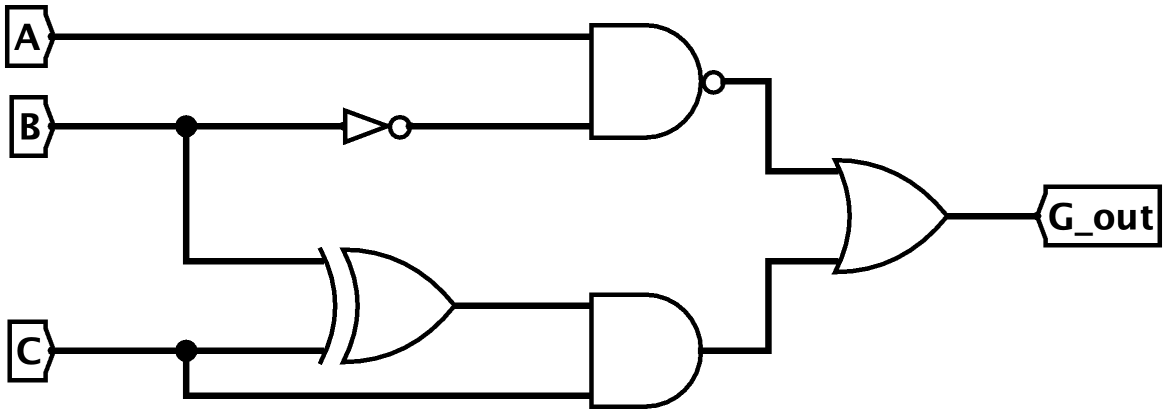
Operators			Symbols		
NOT	AND	OR	Inputs	Constants	Parentheses
$\overline{x}$	$x \cdot y$	$x + y$	A, B, C	0, 1	()

Q7.1 (4 points) Simplify the following boolean algebra expression

$$(\mathrm{A} + \mathrm{C}) \cdot \left(\overline{\mathrm{A}} + \mathrm{B}\right) \cdot \overline{\mathrm{C}}$$

For full credit, reduce the above to an expression that uses at most 3 operators.

Q7.2 (5 points) Give a simplified boolean algebra expression for `G_out` in terms of the inputs `A`, `B` and `C`



For full credit, reduce the circuit above to an expression that uses at most 3 operators.

Q8 *Eggcellence* 🍳 **(0 points)**

These questions will not be assigned credit. Feel free to leave them blank.

Q8.1 Given an egg carton with 12 eggs, what are the first two eggs you pick? (Fill in the square with a 1 for the first egg, and a 2 for the second egg)

LID					
☐ 🐣 A1	☐ 🐣 A2	☐ 🐣 A3	☐ 🐣 A4	☐ 🐣 A5	☐ 🐣 A6
☐ 🐣 B1	☐ 🐣 B2	☐ 🐣 B3	☐ 🐣 B4	☐ 🐣 B5	☐ 🐣 B6

...and why?

Q8.2 If there's anything else you want us to know, or you feel like there was an ambiguity in the exam, please put it in the box below.

For ambiguities, you must qualify your answer and provide an answer for both interpretations. For example, "if the question is asking about A, then my answer is X, but if the question is asking about B, then my answer is Y". You will only receive credit if it is a genuine ambiguity and both of your answers are correct. We will only look at ambiguities if you request a regrade.

Alternatively, draw something egg-cellent!